Lab 06: Parameter Learning with Dirichlet Updates

Objective

Implement online parameter learning, track convergence toward the true model, and evaluate model quality with BMR.

Prerequisites

- Completed Labs 01–05
- Understanding of Dirichlet distributions and conjugate priors

Part 1: Initializing Dirichlet Concentrations

Goal: Set up pA and pB prior concentrations for a 2-state system.

1. Create a `DiscreteEnvironment` with known `true_A` and `true_B`.
2. Initialize `pA = np.ones((2, 2))` (uniform Dirichlet prior).
3. Compute the initial expected A-matrix using `expected_A(pA)`.
4. Compare with `true_A` using KL divergence for each column.

```
 import numpy as np
from active_inference.math import expected_A, kl_divergence

pA = np.ones((2, 2))
A_initial = expected_A(pA)

for s in range(2):
    kl = kl_divergence(A_initial[:, s], true_A[:, s])
    print(f"Initial KL for state {s}: {kl:.4f}")
```

Response:

Write your response here

Part 2: Online Learning Loop

Goal: Run 100 steps of perception-action-learning and track model convergence.

1. Create an agent with the initial expected A-matrix.
2. At each step: infer states, select action, update pA, update pB, recompute expected matrices.
3. Log the KL divergence between `expected_A(pA)` and `true_A` at each step.

```
from active_inference.math import update_dirichlet_A, update_dirichlet_B,

kl_history = []
for t in range(100):
    action = agent.step(obs)
    pA = update_dirichlet_A(pA, obs, agent.q_s, learning_rate=1.0)
    agent.model.A = expected_A(pA)
    obs = env.step(action)
    # TODO: update pB, compute KL, append to kl_history
```

Response:

Write your response here

Part 3: Visualizing Learning Progress

Goal: Plot the learning curve and compare initial vs. learned matrices.

1. Plot KL divergence over time using `plot_learning_progress()`.
2. Plot the initial and final pA using `plot_dirichlet_concentration()`.
3. Plot the final expected A-matrix using `plot_A_matrix()`.

```
from active_inference.visualization import (
    plot_learning_progress, plot_dirichlet_concentration, plot_A_matrix
)

plot_learning_progress(kl_history, save_path="output/lab06_learning.png")
```

Response:

Write your response here

Part 4: Multi-Episode Training

Goal: Run 5 episodes of 50 steps, accumulating pA across episodes.

1. After each episode, reset the environment and agent beliefs, but keep pA.
2. Record the mean KL divergence during each episode.
3. Plot a bar chart of mean KL per episode to show improvement.

Response:

Write your response here

Part 5: Bayesian Model Reduction

Goal: Use BMR to compare the learned model against a reduced model.

1. After 100 steps of learning, compute `bayesian_model_reduction(pA_learned, pA_reduced)`.
2. For `pA_reduced`, set small off-diagonal concentrations to 0.1 (sparser model).
3. Report ΔF and interpret whether the reduced model is preferred.

```
from active_inference.math import bayesian_model_reduction

pA_reduced = pA.copy()
pA_reduced[0, 1] = 0.1  # reduce off-diagonal
pA_reduced[1, 0] = 0.1

delta_F = bayesian_model_reduction(pA, pA_reduced)
print(f"ΔF = {delta_F:.4f} → {'Reduced preferred' if delta_F < 0 else 'Fu
```

Response:

Write your response here

Summary

Skill	Library Component	Status
Initialize Dirichlet concentration priors	`pA = np.ones(...)`, `expected_A()`	
Update pA/pB after each step	`update_dirichlet_A()`, `update_dirichlet_B()`	
Track learning convergence	KL divergence, `plot_learning_progress()`	
Run multi-episode training	Accumulated pA across resets	
Evaluate models with BMR	`bayesian_model_reduction()`	